

Lab 2: Separación de Privilegios y Sandboxing del Lado del Servidor

Curso: IIC2531 - Pontificia Universidad Católica de Chile

Semestre: 2026-1

Número de Lab: Lab 2

Lab 2: Separación de privilegios y sandboxing del lado del servidor

Introducción

Este laboratorio te introducirá a la separación de privilegios y sandboxing del lado del servidor, en el contexto de una aplicación web simple en Python llamada **zoobar**, donde los usuarios transfieren “zoobars” (créditos) entre sí. El objetivo principal de la separación de privilegios es asegurar que si un adversario compromete una parte de la aplicación, el adversario no comprometa las otras partes también. Para ayudarte a separar privilegios en esta aplicación, el servidor web **zookws** está diseñado para ejecutar una aplicación web que consiste en múltiples componentes. Si tienes curiosidad, este diseño está basado en el servidor web OKWS, descrito en un research paper y usado por okcupid.com. En un sistema moderno a gran escala, el diseño probablemente consistiría en muchas más partes móviles, como usar Kubernetes para ejecutar todos los componentes de tu aplicación, usar una librería RPC como gRPC para comunicarse entre componentes, etc, pero **zookws** empaqueta todo esto en un sistema relativamente simple.

En este laboratorio, configurarás un servidor web con separación de privilegios, examinarás posibles vulnerabilidades, y dividirás el código de la aplicación en componentes con menos privilegios para minimizar los efectos de cualquier vulnerabilidad individual. El laboratorio usará soporte moderno para separación de privilegios, Linux containers. (El diseño original de OKWS usaba user IDs y chroot, porque los Linux containers y namespaces no existían en ese momento.)

También extenderás la aplicación web Zoobar para soportar *perfiles ejecutables*, que permiten a los usuarios usar código Python como sus perfiles. Para crear un perfil, un usuario guarda un programa Python en su perfil en su página principal de Zoobar. (Para indicar que el perfil contiene código Python, la primera línea debe ser `#!/python.`) Cada vez que otro usuario vea el perfil Python del usuario, el servidor ejecutará el código Python en ese perfil del usuario para generar la salida del perfil resultante. Esto permitirá a los usuarios implementar una variedad de características en sus perfiles, como:

- Un perfil que saluda a los visitantes por su nombre de usuario.
- Un perfil que mantiene registro de los últimos visitantes a ese perfil.
- Un perfil que da un zoobar a cada visitante (límite 1 por minuto).

Soportar esto de manera segura requiere hacer sandboxing del código del perfil en el servidor, para que no pueda realizar operaciones arbitrarias o acceder a archivos arbitrarios. Por otro lado, este código puede necesitar mantener registro de datos persistentes en algunos archivos, o acceder a bases de datos zoobar existentes, para funcionar correctamente. Usarás la librería de llamadas a procedimiento remoto (RPC) y algún código shim que proporcionamos para hacer sandboxing de manera segura de los perfiles ejecutables en el servidor usando WebAssembly.

Para obtener el código fuente del laboratorio, usa Git para clonar o actualizar el repositorio del lab y cambia a la rama **lab2**.

```
student@6566-v26:~$ cd lab
student@6566-v26:~/lab$ git fetch
...
student@6566-v26:~/lab$ git checkout -b lab2 origin/lab2
Branch lab2 set up to track remote branch lab2 from origin.
Switched to a new branch 'lab2'
```

Una vez que tu código fuente esté en su lugar, asegúrate de que puedas compilar e instalar el servidor web y la aplicación **zoobar**:

```
student@6566-v26:~/lab$ make
cc zookd.c -c -o zookd.o -m64 -g -std=c99 -Wall -Wno-format-overflow -D_GNU_SOURCE -static -fno-stack-protector
cc http.c -c -o http.o -m64 -g -std=c99 -Wall -Wno-format-overflow -D_GNU_SOURCE -static -fno-stack-protector
...
cc zookfs.c -c -o zookfs.o -m64 -g -std=c99 -Wall -Wno-format-overflow -D_GNU_SOURCE -static -fno-stack-protector
cc -m64 zookfs.o http.o -o zookfs
cc zookd2.c -c -o zookd2.o -m64 -g -std=c99 -Wall -Wno-format-overflow -D_GNU_SOURCE -static -fno-stack-protector
cc -m64 zookd2.o http.o -o zookd2
```

```
student@6566-v26:~/lab$
```

Preludio: ¿Qué es un zooobar?

Para entender la aplicación **zooobar** en sí, primero examinaremos el código de la aplicación web **zooobar**.

Una de las características principales de la aplicación **zooobar** es la capacidad de transferir créditos entre usuarios. Esta funcionalidad está implementada por el script **transfer.py**.

Para tener una idea de lo que hace **transfer**, inicia el sitio web **zooobar**. Para el lab 2, iniciamos el web server usando el launcher **zookld.py**, que es similar al daemon launcher **okld** en OKWS:

```
student@6566-v26:~/lab$ ./zookld.py
~base: Creating container
Unpacking the rootfs

---
You just created an Ubuntu noble amd64 (20260126_07:42) container.

To enable SSH, run: apt install openssh-server
No default root or user password are set by LXC.
~base: Configuring
... lots of output ...
main: Creating container
main: Copying files
main: Running zooksvc.py
main: zooksvc.py: dispatcher main, port 8080
main: zooksvc.py: running ['./zookd2', '3', '5']
main: zookd2: Start with 1 service(s)
main: zookd2: Dispatch ^(.*)$ for service 0
main: zookd2: Host 10.1.1.4 (link 1) service 0
main: zookd2: Port 8081 for service 0
zookfs: Creating container
zookfs: Copying files
zookfs: Running zooksvc.py
zookfs: zooksvc.py: running ['./zookfs', '8081']
echo: Creating container
echo: Copying files
echo: Running zooksvc.py
echo: zooksvc.py: running ['./zooobar/echo-server.py', '8081']
echo: Running on port 8081
student@6566-v26:~/lab$
```

La primera vez que ejecutes **zookld.py** tardará minutos, porque está construyendo el container **base**, lo que implica construir una imagen de Linux e instalar todo el software que **zooobar** necesita. Luego, construye otros tres containers: **main**, **zookfs**, y **echo**. Cada uno de estos containers tiene el contenido de tu directorio **/home/student/lab** copiado en **/app**, para que puedas ejecutar tu código dentro del container. Los containers mismos se almacenan en el directorio **~/local/share/lxc**, si tienes curiosidad sobre cómo están implementados los containers. **zookld.py** usa **zookconf.py** para construir y configurar los containers. Si obtienes errores sobre crear o iniciar containers, intenta reiniciar el estado del container usando el comando **./zookclean.py** y/o reiniciando tu VM.

Puedes manipular los containers con los siguientes comandos:

- **zookld.py**: inicia los containers listados en **zook.conf**
- **zookps.py**: lista el estado de los containers listados en **zook.conf** y los procesos ejecutándose en ellos.
- **zookstop.py**: detiene los containers listados en **zook.conf**
- **zookclean.py**: elimina todos los containers de **~/local/share/lxc**, por si quieres empezar de cero nuevamente.

Cada uno de estos comandos también puede recibir el nombre de un container individual (ej., **main**) y aplicar la operación solo a ese container.

Todos estos comandos son wrappers alrededor de la API de Python de LXC. También puedes ejecutar muchos de los comandos de LXC desde línea de comandos; en particular, **lxc-attach -n name** te dará una shell de root en el container **name**, lo cual

podrías encontrar útil para depuración.

Ejecuta `zookps.py` para ver si tus containers están corriendo. Luego, asegúrate de que puedes ejecutar el web server, y acceder al sitio web desde tu navegador, de la siguiente manera:

```
student@65566-v26:~/lab$ ip addr show dev eth0
2: eth0: mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:b4:55:8e brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    altname ens3
    inet 192.168.24.128/24 brd 192.168.24.255 scope global eth0
    valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feb4:558e/64 scope link
    valid_lft forever preferred_lft forever
```

En este ejemplo particular, querías abrir tu navegador e ir a <http://192.168.24.128:8888/zoobar/index.cgi/>, o, si estás usando KVM, a <http://localhost:8888/zoobar/index.cgi/>. Deberías ver el sitio web de zoobar.

Nota el puerto diferente, 8888 en lugar de 8080, en las URLs de arriba. Esto es porque en realidad queremos conectarnos al container `main`, pero está en una virtual network interna que solo es alcanzable desde la VM misma. Configuramos el kernel de Linux en la VM para reenviar conexiones al puerto 8888 hacia el puerto 8080 en el container `main`. Puedes ver las reglas de `iptables` que usamos para lograr esto en `/etc/rc.local` en tu VM.

Si tienes problemas para ver el sitio web, verifica que la configuración de tu web server no sea demasiado restrictiva.

Ejercicio 1. En tu navegador, conéctate al sitio Web de **zoobar**, y crea dos cuentas de usuario. Inicia sesión como uno de los usuarios, y transfiere zoobars de un usuario a otro haciendo clic en el enlace de transferencia y llenando el formulario. Juega también con las otras características para tener una idea de lo que permite hacer a los usuarios. En resumen, un usuario registrado puede actualizar su perfil, transferir “zoobars” (créditos) a otro usuario, y consultar el balance de zoobars, perfil, y transacciones de otros usuarios en el sistema.

Lee el código de **zoobar** y observa cómo `transfer.py` es invocado cuando un usuario envía una transferencia en la página de transferencia. Un buen lugar para comenzar en esta parte del laboratorio es `templates/transfer.html`, `__init__.py`, `transfer.py`, y `bank.py` en el directorio **zoobar**.

Nota: No necesitas entregar nada para este ejercicio, ¡pero asegúrate de que entiendes la estructura de la aplicación zoobar—te ahorrará tiempo en el futuro!

Separación de privilegios

Habiendo revisado el código de la aplicación zoobar, vale la pena comenzar a pensar sobre cómo aplicar separación de privilegios a la infraestructura de **zookws** y **zoobar** para que los errores en la infraestructura no permitan a un adversario, por ejemplo, transferir zoobars a la cuenta del adversario.

El web server para este laboratorio usa containers para diferentes partes del web server. Los containers están definidos en `zook.conf`. `zookld.py` usa `zookconf.py` para leer `zook.conf` (mediante `readconf.py`) y configura los containers. Como parte de este laboratorio definirás nuevos containers y modificarás cómo se configuran los containers. Para ese propósito necesitas trabajar con `LXC` y puede resultarte útil consultar la documentación de `LXC`.

Dos aspectos hacen que la separación de privilegios sea desafiante en el mundo real y en este laboratorio. Primero, la separación de privilegios requiere que desarmes la aplicación y la dividas en piezas separadas. Aunque hemos intentado estructurar bien la aplicación para que sea fácil de dividir, hay lugares donde debes rediseñar ciertas partes para hacer posible la separación de privilegios. Segundo, debes asegurar que cada pieza se ejecute con privilegios mínimos, lo que requiere establecer permisos precisamente y configurar las piezas correctamente. Con suerte, al final de este laboratorio, tendrás una mejor comprensión de por qué muchas aplicaciones tienen vulnerabilidades de seguridad relacionadas con fallos en separar apropiadamente los privilegios: ¡la separación de privilegios adecuada es difícil!

Un problema que podrías encontrar es que es complicado depurar una aplicación compleja que está compuesta de muchas piezas. Para ayudarte, hemos proporcionado una biblioteca de depuración simple en `debug.py`, que es importada por cada script de Python que te damos. La biblioteca `debug` proporciona una única función, `log(msg)`, que imprime el mensaje `msg` a `stderr` (que debería ir a la terminal donde ejecutaste `zookld`), junto con un rastreo de pila de dónde se llamó la función `log`.

Toda la salida del código que se ejecuta en varios containers que ves en tu terminal también debería estar prefijada con el nombre del container. Por ejemplo, cuando ves `main: zookd2: Forwarding to 10.1.1.4:8081 for /zoobar/media/zoobar.css`, significa que el mensaje viene del container `main`.

Si algo no parece estar funcionando, intenta descubrir qué salió mal, o contacta al instructor del curso o los ayudantes, antes de continuar.

Parte 1: Separar privilegios en la configuración del web server usando containers

Anteriormente, **zookws** consistía esencialmente en un proceso: **zookd**. Desde el punto de vista de seguridad, esta estructura no es ideal: por ejemplo, cualquier desbordamiento de búfer podría usarse para tomar control de **zookws**. Por ejemplo, puedes invocar los scripts dinámicos con argumentos de tu elección (ej., darte muchos zoobars a ti mismo), o más simple, simplemente escribir la base de datos que contiene las cuentas de zoobar directamente.

Este laboratorio refactoriza **zookd** siguiendo el diseño de OKWS. Similar a OKWS, **zookws** consiste en un programa lanzador **zookld.py** que inicia servicios configurados en el archivo **zook.conf**, un **zookd** que solo enruta solicitudes a los servicios correspondientes, así como varios servicios. Por simplicidad **zookws** no implementa daemons auxiliares o de registro como lo hace OKWS. Puedes pensar en **zookld.py** como una versión minimalista de Kubernetes.

Ejecutaremos cada componente en un container Linux separado. Los containers Linux proporcionan la ilusión de una máquina Linux virtual sin usar máquinas virtuales; están implementados usando procesos Linux. Un proceso en un container está más fuertemente aislado que los procesos Linux estándar: un proceso dentro de un container tiene acceso limitado a espacios de nombres del kernel, tiene acceso limitado a llamadas al sistema, y no tiene acceso al sistema de archivos. En muchos sentidos, se comportan como máquinas virtuales: se inician desde una imagen de VM, tienen su propia dirección IP, su propio sistema de archivos, etc. Les asignas direcciones IP, copias los archivos correctos en ellos, y organizas llamadas de procedimiento remoto entre ellos.

Usaremos containers *no privilegiados*, que se ejecutan como un proceso de usuario no privilegiado (es decir, no root). Incluso si el proceso que se ejecuta dentro del container se ejecuta con privilegios de root, el container mismo se ejecuta como un usuario no privilegiado.

Con containers, incluso si hay un exploit (ej., otro desbordamiento de búfer en **zookd**), el container ejecutando **zookd** le dará al atacante poco control. Por ejemplo, tomar control de **zookd**, no permitirá al atacante invocar los scripts dinámicos o escribir directamente la base de datos que se ejecuta dentro de otro container. Además, **zookd** no puede escapar de su container y tomar control sobre el kernel Linux subyacente.

El archivo **zook.conf** es el archivo de configuración que especifica cómo debe ejecutarse cada container. Por ejemplo, la entrada **main**:

```
[main]
cmd = zookd2
dir = /app
lxcbr = 0
port = 8080
http_svcs = zookfs
```

especifica que el comando para ejecutar **main** es **zookd2**, en el directorio **/app** en un container conectado a la virtual network 0 (**lxcbr**, abreviatura de LXC bridge), y que **cmd** obtiene el puerto 8080 para recibir/enviar solicitudes.

En la VM del laboratorio que estás usando, pre-creamos 10 virtual networks, **lxcbr0** hasta **lxcbr9**, que puedes usar, correspondientes a las direcciones de red 10.1.0.* hasta 10.1.9.* respectivamente (o 10.1.0.0/24 hasta 10.1.9.0/24 en notación CIDR). Cada container obtiene la dirección IP .4 en la subred de su virtual network; por ejemplo, si asignas algún servicio a **lxcbr = 7**, tendrá la dirección IP 10.1.7.4.

La razón de tener múltiples virtual networks es proporcionar un fuerte aislamiento de red entre containers. Los containers conectados a la misma virtual network pueden enviarse paquetes directamente, y un container comprometido podría falsificar paquetes desde la dirección IP de otro container. Por otro lado, los containers en diferentes virtual networks deben pasar por el kernel de Linux del host para enrutar paquetes, y el kernel de Linux asegura que los paquetes provenientes de una virtual network no falsifiquen una dirección de origen de una virtual network diferente (habilitado por la opción del kernel **rp_filter**).

Por defecto, cada container permite paquetes entrantes de cualquier otro container. Restringirás esto más adelante en el laboratorio, y el uso de virtual networks separadas asegurará que un container comprometido no pueda evadir estas restricciones a nivel de red.

El archivo **zook.conf** configura solo un servicio HTTP (a través de la línea **http_svcs**), **zookfs**, que tanto sirve archivos estáticos como ejecuta scripts dinámicos. Más adelante en este laboratorio, necesitarás ejecutar múltiples servicios HTTP, lo cual puedes hacer listándolos todos, separados por coma, en la línea **http_svcs**; por ejemplo, **http_svcs = first,second,third**.

El servicio **zookfs** funciona invocando el ejecutable **zookfs**, que corre en el directorio **/app** en el container conectado a **lxcbr = 1** (y por lo tanto con dirección IP 10.1.1.4).

Las líneas **fwrule** que ves comentadas en la descripción del servicio **zookfs** en **zook.conf** especifican filtros que controlan la comunicación hacia ese container:

```
[zookfs]
cmd = zookfs
url = .*
dir = /app
lxcbr = 1
port = 8081
## Filter rules are inserted in the order they appear in this file.
## Thus, in the below example (commented out initially) the first
## filters applied are the ACCEPT ones, and then the REJECT one.
## Use `iptables -nvL INPUT' on the appropriate container to see all
## the filters that are in effect on that container.
# fwrule = -s main -j ACCEPT
# fwrule = -s echo -j ACCEPT
# fwrule = -j REJECT
```

Por ejemplo, las reglas comentadas permiten paquetes de los containers **main** y **echo**, pero bloquean todos los demás paquetes. Cambiarás estas reglas (y definirás reglas similares para otros containers) a medida que sepamos más privilegios de **zookws**.

Comenzaremos a separar más privilegios del servicio **zookfs** que maneja tanto archivos estáticos como scripts dinámicos. Aunque corre en un container, algunos scripts de Python podrían fácilmente tener vulnerabilidades de seguridad; un script de Python vulnerable podría ser engañado para eliminar archivos estáticos importantes que el server está sirviendo. Por el contrario, el código de servicio de archivos estáticos podría ser engañado para servir las bases de datos usadas por los scripts de Python, como **person.db** y **transfer.db**. Una mejor organización es dividir **zookfs** en dos servicios, uno para archivos estáticos y otro para scripts de Python, ejecutándose como diferentes usuarios.

Ejercicio 2. Modifica **zook.conf** para reemplazar **zookfs** con dos servicios separados, **dynamic** y **static**. Ambos deben usar **cmd = zookfs**.

dynamic debe ejecutar solo **/zoobar/index.cgi** (que ejecuta todos los scripts de Python), pero no debe servir ningún archivo estático. **static** debe servir archivos estáticos pero no ejecutar nada.

Para eliminar el container **zookfs** para que **zookld.py** no lo inicie, ejecuta **./zookclean.py zookfs**.

Ejecuta los servicios **dynamic** y **static** en diferentes virtual networks.

Esta separación requiere que **zookd** determine qué servicio debe manejar una solicitud particular. Puedes usar el filtrado de URL de **zookws** para hacer esto, sin modificar la aplicación ni las URLs que usa. Los filtros de URL se especifican en **zook.conf**, y soportan expresiones regulares. Por ejemplo, **url = .*** coincide con todas las solicitudes, mientras que **url = /zoobar/(abc|def)\.html** coincide con solicitudes a **/zoobar/abc.html** y **/zoobar/def.html**.

Para este ejercicio, solo debes modificar **zook.conf**; no modifiques ningún código C o Python.

Ejecuta **make check-lab2** para verificar que tu configuración modificada pasa nuestras pruebas.

Nos gustaría controlar con quién pueden comunicarse **dynamic** y **static**. Por ejemplo, incluso si **static** fuera comprometido, queremos que el atacante sea incapaz de comunicarse con **dynamic**, haciendo más difícil para el atacante comprometer **dynamic** también. Por lo tanto, nos gustaría configurar reglas de firewall en cada container para hacer cumplir este aislamiento. Usaremos **iptables** para insertar reglas de filtro en cada container, de modo que **static** solo acepte paquetes de **main** (que ejecuta **zookd**). Esto asegura que **dynamic** no pueda enviar paquetes directamente a **static**. De manera similar, **dynamic** solo debe aceptar paquetes de **main**, y no de **static**. Configurar reglas de seguridad como estas a menudo se denomina segregación de red.

Ejercicio 3. Escribe entradas **fwrule** apropiadas para **main**, **static**, y **dynamic** para limitar la comunicación como se especificó anteriormente.

Si configuras los filtros incorrectamente, podrías ser incapaz de conectarte con algún container. Puedes restablecer el firewall para permitir toda comunicación deteniendo e iniciando los containers nuevamente, usando **zookstop.py** seguido de **zookld.py**. ## Interludio: Biblioteca RPC

En esta parte, separarás por privilegios la aplicación **zoobar** en varios procesos, ejecutándose en diferentes containers. Nos gustaría limitar el daño de cualquier bug futuro que aparezca. Es decir, si una parte de la aplicación **zoobar** tiene un bug explotable, nos gustaría prevenir que un atacante use ese bug para comprometer otras partes de la aplicación **zoobar**.

Un desafío al dividir la aplicación **zoobar** en varios containers es que los procesos dentro de los containers deben tener una forma de comunicarse entre sí. Primero estudiarás una biblioteca de Remote Procedure Call (RPC) que permite que los procesos se comuniquen. Luego, usarás esa biblioteca para separar **zoobar** en varios procesos, cada uno dentro de su propio container, que se comunican usando RPC.

Para ilustrar cómo podría usarse nuestra biblioteca RPC, hemos implementado un simple servicio “echo” para ti, en **zoobar/echo-server.py**. Este servicio es invocado por **zookld** y se ejecuta dentro de su propio container; busca la sección **echo** de **zook.conf** para ver cómo se inicia.

echo-server.py está implementado definiendo una clase RPC **EchoRpcServer** que hereda de **RpcServer**, que a su vez viene de **zoobar/rplib.py**. La clase RPC **EchoRpcServer** define los métodos que el server soporta, y **rplib** invoca esos métodos cuando un cliente envía una solicitud. El server define un método simple que hace eco de la solicitud de un cliente.

echo-server.py inicia el server llamando a **run_fork(port)**. Esta función escucha en un socket TCP. El puerto viene del argumento, que en este caso es **8081** (especificado en **zook.conf**). Cuando un cliente se conecta a este socket, la función hace fork del proceso actual. Una copia del proceso recibe mensajes y responde en la conexión recién abierta, mientras que el otro proceso escucha por otros clientes que podrían abrir el socket.

También hemos incluido un cliente simple de este servicio **echo** como parte de la aplicación web Zoobar. En particular, si vas a la URL **/zoobar/index.cgi/echo?s=hello**, la solicitud es enrutada a **zoobar/echo.py**. Ese código usa el cliente RPC (implementado por **rplib**) para conectarse al servicio echo en (dirección IP del host, puerto). El cliente busca la (dirección IP del host, puerto) en **zook.conf**. El cliente invoca la operación **echo**. Una vez que recibe la respuesta del servicio echo, devuelve una página web que contiene la respuesta en eco.

El código del lado cliente RPC en **rplib** está implementado por el método **call** de la clase **RpcClient**. Este método formatea los argumentos en un string, escribe el string en la conexión al server, y espera una respuesta (un string). Al recibir la respuesta, **call** parsea el string, y devuelve los resultados al llamador.

Parte 2: Separación de privilegios del servicio de login en Zoobar

Ahora usaremos la biblioteca RPC para mejorar la seguridad de las contraseñas de usuario almacenadas en la aplicación web Zoobar. Ahora mismo, un adversario que explota una vulnerabilidad en cualquier parte de la aplicación Zoobar puede obtener todas las contraseñas de usuario de la base de datos **person**.

El primer paso para proteger las contraseñas será crear un servicio que maneje las contraseñas de usuario y cookies, de modo que solo ese servicio pueda acceder a ellas directamente, y el resto de la aplicación Zoobar no pueda. En particular, queremos separar el código que maneja la autenticación de usuarios (es decir, contraseñas y tokens) del resto del código de la aplicación. La aplicación **zoobar** actual almacena todo sobre el usuario (su perfil, su saldo de zoobar, e información de autenticación) en la tabla **Person** (ver **zodb.py**). Queremos mover la información de autenticación fuera de la tabla **Person** a una tabla **Cred** separada (**Cred** significa Credenciales), y mover el código que accede a esta información de autenticación (es decir, **auth.py**) a un servicio separado.

Ten en cuenta que no es completamente necesario dividir los datos en tablas separadas por seguridad: cada container terminaría con su propia copia de la base de datos, y no tendría datos en las partes de la base de datos que nunca pobló. Dividimos los datos en tablas separadas de todos modos, porque ayuda a entender cómo dividir los servicios correctamente.

Específicamente, tu trabajo será el siguiente:

- Decidir qué interfaz debería proporcionar tu servicio de autenticación (es decir, qué funciones ejecutará para los clientes). Mira el código en **login.py** y **auth.py**, y decide qué necesita ejecutarse en el servicio de autenticación, y qué puede ejecutarse en el cliente (es decir, ser parte del resto del código de zoobar). Ten en cuenta que tu objetivo es proteger tanto las contraseñas como los tokens. Hemos proporcionado stubs RPC iniciales para el cliente en el archivo **zoobar/auth_client.py**.
- Crear un nuevo servicio **auth** para autenticación de usuarios, siguiendo la línea de **echo-server.py**. Hemos proporcionado un archivo inicial para ti, **zoobar/auth-server.py**, que deberías modificar para este propósito. La implementación de este servicio debería usar las funciones existentes en **auth.py**.
- Modificar **zook.conf** para iniciar el **auth-server** apropiadamente (en un container en una virtual network diferente).
- Separar las credenciales de usuario (es decir, contraseñas y tokens) de la base de datos **Person** a una base de datos **Cred** separada, almacenada en **/zoobar/db/cred**. No mantengas ninguna contraseña o token en la antigua base de datos **Person**.

- Modificar el código de login en `login.py` para invocar tu servicio `auth` en lugar de llamar a `auth.py` directamente.

Ejercicio 4. Implementa la separación de privilegios para la autenticación de usuarios, como se describe arriba.

Un buen punto de partida sería primero separar las credenciales originalmente almacenadas en la base de datos `Person` en una base de datos `Cred` separada, pero aún hacer todo en el servicio `dynamic`. No olvides crear una entrada regular en la base de datos `Person` para los usuarios recién registrados.

Una vez que eso funcione, añade llamadas de función explícitas entre el código que esperas que aún viva en el servicio `dynamic` (que no toca `Cred`) y las funciones que eventualmente moverás a `auth-server` (que sí toca `Cred`). Asegúrate de que esto funcione con solo funciones primero, sin realmente mover el código a un `auth-server` separado.

Finalmente, configura el `auth-server.py` para ejecutar el código que maneja la base de datos `Cred`, y convierte las llamadas de función del paso anterior en RPCs reales a `auth-server`.

Ejecuta `make check-lab2` para verificar que tu servicio de autenticación con separación de privilegios pasa nuestras pruebas.

Ejercicio 5. Especifica las entradas `fwrule` apropiadas para `auth`.

Ahora, mejoraremos aún más la seguridad de las contraseñas, usando hashing y salting. El código de autenticación actual almacena una copia exacta de la contraseña del usuario en la base de datos. Por lo tanto, si un adversario de alguna manera obtiene acceso al archivo `cred.db`, todas las contraseñas de usuario serán comprometidas inmediatamente. ¡Peor aún, si los usuarios tienen la misma contraseña en múltiples sitios, el adversario podrá comprometer las cuentas de los usuarios allí también!

El hashing protege contra este ataque, almacenando un hash de la contraseña del usuario (es decir, el resultado de aplicar una función hash a la contraseña), en lugar de la contraseña en sí. Si la función hash es difícil de invertir (es decir, es un hash criptográficamente seguro), un adversario no podrá obtener directamente la contraseña del usuario. Sin embargo, un server aún puede verificar si un usuario proporcionó la contraseña correcta durante el login: simplemente calculará el hash de la contraseña del usuario, y verificará si el valor hash resultante es el mismo que se almacenó previamente.

Una debilidad del hashing es que un adversario puede construir una tabla gigante (llamada “tabla rainbow”), que contiene los hashes de todas las contraseñas posibles. Entonces, si un adversario obtiene la contraseña hasheada de alguien, el adversario puede simplemente buscarla en su tabla gigante, y obtener la contraseña original.

Para derrotar el ataque de tabla rainbow, la mayoría de los sistemas usan *salting*. Con salting, en lugar de almacenar un hash de la contraseña, el server almacena un hash de la contraseña concatenada con un string generado aleatoriamente (llamado salt). Para verificar si la contraseña es correcta, el server concatena la contraseña proporcionada por el usuario con el salt, y verifica si el resultado coincide con el hash almacenado. ¡Ten en cuenta que, para que esto funcione, el server debe almacenar el valor del salt usado para calcular originalmente el hash con salt! Sin embargo, debido al salt, el adversario ahora tendría que generar una tabla rainbow separada para cada valor de salt posible. Esto aumenta enormemente la cantidad de trabajo que el adversario tiene que realizar para adivinar las contraseñas de los usuarios basándose en los hashes.

Una consideración final es la elección de la función hash. La mayoría de las funciones hash, como MD5 y SHA1, están diseñadas para ser rápidas. Esto significa que un adversario puede probar muchas contraseñas en un corto período de tiempo, ¡lo cual no es lo que queremos! En cambio, deberías usar una función especial tipo hash que está explícitamente diseñada para ser *lenta*. Un buen ejemplo de tal función hash es PBKDF2, que significa Password-Based Key Derivation Function (versión 2).

Ejercicio 6. Implementa hashing y salting de contraseñas en tu servicio de autenticación. En particular, necesitarás extender tu tabla `Cred` para incluir una columna `salt`; modificar el código de registro para elegir un salt aleatorio, y almacenar un hash de la contraseña junto con el salt, en lugar de la contraseña en sí; y modificar el código de login para calcular el hash de la contraseña proporcionada junto con el salt almacenado, y compararla con el hash almacenado. No elimines la columna de contraseña de la tabla `Cred` (la verificación del ejercicio 5 requiere que esté presente); puedes almacenar la contraseña hasheada en la columna `password` existente.

Para implementar hashing con PBKDF2, puedes usar el módulo Python PBKDF2. A grandes rasgos, deberías hacer `import pbkdf2`, y luego calcular el hash de una contraseña usando `pbkdf2.PBKDF2(password, salt).hexread(32)`. Hemos proporcionado una copia de `pbkdf2.py` en el directorio `zooar`. No uses la función `random.random` para generar un salt ya que la documentación del módulo `random` indica que no es criptográficamente seguro. Una alternativa segura es la función `os.urandom`.

Ejecuta `make check-lab2` para verificar que tu código de hashing y salting pasa nuestras pruebas. Ten en cuenta que nuestras pruebas no son exhaustivas.

Un efecto secundario sorprendente de usar una función hash computacionalmente muy costosa como PBKDF2 es que un adversario ahora puede usar esto para lanzar ataques de denegación de servicio (DoS) en la CPU del server. Por ejemplo, el popular framework web Django publicó un aviso de seguridad sobre esto, señalando que si un adversario intenta iniciar sesión en alguna cuenta proporcionando una contraseña muy grande (1MB de tamaño), el server pasaría un minuto entero tratando de calcular PBKDF2 sobre esa contraseña. La solución de Django es limitar las contraseñas proporcionadas a un máximo de 4KB de tamaño. Para este laboratorio, no requerimos que manejes tales ataques DoS.

Parte 3: Separación de privilegios del banco en Zoobar

Finalmente, queremos proteger el saldo de zoobars de cada usuario de adversarios que podrían explotar algún bug en la aplicación Zoobar. Actualmente, si un adversario explota un bug en la aplicación principal de Zoobar, pueden robar los zoobars de cualquier otra persona, y esto ni siquiera aparecería en la base de datos **Transfer** si quisiéramos auditar las cosas más tarde.

Para mejorar la seguridad de los saldos de zoobar, nuestro plan es similar a lo que hiciste anteriormente en el servicio de autenticación: separar la información del saldo **zoobar** en una base de datos **Bank** separada, y configurar un servicio **bank**, cuyo trabajo es realizar operaciones en la nueva base de datos **Bank** y la base de datos **Transfer** existente. Mientras solo el servicio **bank** pueda modificar las bases de datos **Bank** y **Transfer**, los bugs en el resto de la aplicación Zoobar no deberían dar a un adversario la capacidad de modificar saldos de zoobar, y asegurarán que todas las transferencias se registren correctamente para futuras auditorías.

Ejercicio 7. Separa por privilegios la lógica del banco en un servicio **bank** separado, siguiendo las líneas del servicio de autenticación. Tu servicio debe implementar las funciones **transfer** y **balance**, que actualmente están implementadas por **bank.py** y se llaman desde varios lugares en el resto del código de la aplicación.

Debes separar la información del saldo **zoobar** en una base de datos **Bank** separada (en **zoodb.py**); implementar el server del banco modificando **bank-server.py**; agregar el servicio bank a **zook.conf**; crear stubs de cliente RPC para invocar el servicio bank; y modificar el resto del código de la aplicación para invocar los stubs RPC en lugar de llamar directamente a las funciones de **bank.py**.

No olvides manejar el caso de creación de cuenta, cuando el nuevo usuario necesita obtener 10 zoobars iniciales. Esto puede requerir que cambies la interfaz del servicio bank.

Ejecuta `make check-lab2` para verificar que tu servicio bank con separación de privilegios pasa nuestras pruebas.

Finalmente, necesitamos solucionar un problema más con el servicio bank. En particular, un adversario que puede acceder al servicio bank (es decir, puede enviarle solicitudes RPC) puede realizar transferencias desde la cuenta de *cualquiera* a la suya propia. Por ejemplo, puede robar 1 zoobar de cualquier víctima simplemente emitiendo una solicitud RPC **transfer(victim, adversary, 1)**. El problema es que el servicio bank no tiene idea de quién está invocando la operación **transfer**. Algunas bibliotecas RPC proporcionan autenticación, pero nuestra biblioteca RPC es bastante simple, así que tenemos que agregarla explícitamente.

Para autenticar al llamador de la operación **transfer**, requeriremos que el llamador proporcione un argumento **token** adicional, que debe ser un token válido para el remitente. El servicio bank debe rechazar transferencias si el token es inválido.

Ejercicio 8. Agrega autenticación al RPC **transfer** en el servicio bank. El token del usuario actual es accesible como **g.user.token**. ¿Cómo debería el banco validar el token proporcionado?

Ejercicio 9. Especifica las entradas **fwrule** apropiadas para el servicio **bank**.

Parte 4: Sandboxing del lado del servidor para perfiles ejecutables

En esta parte final, implementarás sandboxing para perfiles ejecutables, lo cual es desafiante porque necesitamos aislar los perfiles de diferentes usuarios entre sí y del resto del sistema.

Como ejemplo, aquí está el tipo de funcionalidad que nos gustaría soportar con perfiles ejecutables. Un usuario podría establecer su perfil con el siguiente código (que puedes encontrar en **profiles/hello-user.py**):


```
#!/python
import time
import api
print('Hello, *', api.call('get_visitor'), '*')
print('
Current time:', time.time())
```

Cuando alguien ve el perfil de este usuario en Zoobar, el server ejecutará este código Python, y mostrará la salida resultante como si ese fuera el perfil del usuario. Por ejemplo, si la usuaria `alice` ve este perfil, podría ver el siguiente resultado en su pantalla:

```
Hello, alice
```

```
Current time: 1708447443.7460613
```

La implementación inicial que proporcionamos para perfiles ejecutables ejecuta el código del perfil directamente en el proceso Python del server, lo que significa que un usuario malicioso podría manipular el server inyectando código arbitrario a través de su perfil.

Usaremos WebAssembly para ejecutar el perfil ejecutable de un usuario en aislamiento, de modo que el perfil ejecutable de un usuario no pueda manipular los perfiles de otros usuarios. WebAssembly es útil aquí porque hace que crear un nuevo entorno de ejecución aislado sea ligero, y la creación puede realizarse fácilmente bajo demanda, en contraste con los containers (crear containers es algo intensivo en recursos y requiere privilegios especiales que no están disponibles para un container mismo).

Para aislar código Python arbitrario usando WebAssembly, te hemos proporcionado una versión del intérprete de Python que puede ejecutarse dentro de un módulo WebAssembly. Luego ejecutaremos este intérprete de Python completo, más el código de perfil Python proporcionado por el usuario, dentro del módulo WebAssembly. Esta versión de Python está instalada como `/usr/local/share/python-3.12.0.wasm` dentro de tu VM.

Si quieres tener una idea de este intérprete de Python aislado en WebAssembly, puedes ejecutarlo desde la línea de comandos en tu VM de la siguiente manera:

```
student@6566-v26:~/lab$ wasmtime run -- /usr/local/share/python-3.12.0.wasm
Python 3.12.0 (tags/v3.12.0:0fb18b0, Dec 11 2023, 11:45:20) [Clang 16.0.0 ] on wasi
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

El comando anterior usa el runtime WebAssembly `wasmtime` para ejecutar el intérprete de Python desde `/usr/local/share/python-3`

Dado que los perfiles ejecutables son una parte algo peligrosa del diseño general del sistema (estamos ejecutando código arbitrario, después de todo), estaremos ejecutando estos perfiles ejecutables en un container separado dedicado a ese propósito, muy similar a las partes anteriores de este laboratorio. Esto no aísla los perfiles entre sí, pero al menos proporciona algún grado de aislamiento entre todos los perfiles ejecutables y el resto del sistema.

Debes familiarizarte con el directorio `profiles/`, que contiene varios perfiles ejecutables que usarás como ejemplos:

- `profiles/hello-user.py` es un perfil simple que imprime el nombre del visitante cuando se ejecuta el código del perfil, junto con la hora actual.
- `profiles/visit-tracker.py` mantiene un registro de la última vez que cada visitante miró el perfil, e imprime la hora de la última visita (si existe).
- `profiles/last-visits.py` registra los últimos tres visitantes al perfil, y los imprime.
- `profiles/xfer-tracker.py` imprime la última transferencia de zoobar entre el propietario del perfil y el visitante.
- `profiles/granter.py` le da al visitante un zoobar. Para asegurar que los visitantes no puedan robar rápidamente todos los zoobars de un usuario, este perfil otorga un zoobar solo si el propietario del perfil tiene algunos zoobars restantes, el visitante tiene menos de 20 zoobars, y ha pasado al menos un minuto desde la última vez que ese visitante obtuvo un zoobar de este perfil.

La pieza final de código es `zoobar/profile-server.py`: un server RPC que acepta solicitudes para ejecutar el código del perfil de algún usuario, y devuelve la salida de ejecutar ese código. Al ejecutar un perfil, `profile-server.py` configura un server RPC que permite que el código del perfil obtenga acceso a cosas fuera del sandbox, como los saldos de zoobar de diferentes usuarios. El `ProfileAPIServer` implementa esta interfaz; `profile-server.py` bifurca un proceso separado para ejecutar el `ProfileAPIServer`.

Un desafío es que el servicio `profile` realiza `rpc_xfer` desde la cuenta del propietario del perfil, lo cual (dependiendo de tu diseño para la separación de privilegios del banco en el ejercicio 8 anterior) puede requerir un token para el propietario. No puedes simplemente agregar un RPC al servicio `auth` para obtener un token para el propietario del perfil, porque entonces cualquier servicio podría pedirlo; queremos que solo el servicio `profile` pueda hacer esta transferencia. De manera similar, no podemos agregar un RPC al servicio `bank` para hacer una transferencia desde la cuenta de cualquiera sin un token, ya que eso rompería las garantías de seguridad del ejercicio 8.

Para los propósitos de este laboratorio, queremos hacer la transferencia desde la cuenta del propietario del perfil solo si la solicitud vino del servicio `profile`. Por lo tanto, el servicio `bank` debe poder autenticar que una solicitud vino del servicio `profile`. Para ayudarte a hacer esto, `rpcsrv.py` proporciona el nombre del servicio que llama en `self.caller`, basado en la dirección IP de la conexión desde la cual recibió la solicitud.

Para comenzar, necesitarás agregar el servicio `profile` a tu `zook.conf`. Ejecuta el servicio en su propio container y en su propio enlace de red.

Ejercicio 10. Agrega `profile-server.py` a tu web server, e implementa la lógica en `ProfileServer.rpc_run()` para ejecutar correctamente perfiles ejecutables en un sandbox WebAssembly. Puede que te sea útil referirte a la documentación de los bindings de Python para wasmtime. También hay varias sugerencias en los comentarios en `profile-server.py`.

Asegúrate de que tu sitio Zoobar pueda soportar los cinco perfiles.

Ejecuta `make check-lab2` para verificar que tu configuración modificada pasa nuestras pruebas. El caso de prueba crea algunas cuentas de usuario, almacena uno de los perfiles de Python en el perfil de un usuario, hace que otro usuario vea ese perfil, y verifica que el otro usuario vea la salida correcta.

Si encuentras problemas con las pruebas de `make check-lab2`, puedes revisar `/tmp/html.out` para la salida html de los perfiles.

El diseño anterior puede tener varios problemas de seguridad. Primero, es importante que el código del perfil en sandbox no pueda manipular el estado de los perfiles de otros usuarios. Segundo, es importante que un perfil malicioso no pueda ejecutarse en bucle infinitamente.

Ejercicio 11. Modifica aún más `rpc_run` en `profile-server.py` para que el sandbox evite que el código del perfil se ejecute por más de 5 segundos.

Ejecuta `make check-lab2` para ver si tu implementación pasa nuestros casos de prueba.

Ejercicio 12. Especifica las entradas `fwrule` apropiadas para `profile` y otros servicios. Por ejemplo, `profile` no debería poder comunicarse con `auth`.

Ahora has terminado con el sandbox básico.

¡Has terminado! Ejecuta `make handin.zip` y sube el archivo `handin.zip` resultante a Canvas.

Agradecimientos

Gracias al equipo del curso CS155 de Stanford por el código inicial de la aplicación web zoobar, que extendimos en esta tarea de laboratorio.

Basado en: MIT Course 6.566 Lab 2 (Spring 2026)

URL Original: <https://css.csail.mit.edu/6.566/2026/labs/lab2.html>

Licencia: Creative Commons Attribution 3.0 Unported

Este laboratorio está adaptado de los materiales del curso Computer Systems Security del MIT. Todo el crédito del contenido original corresponde al equipo docente de MIT CSAIL.